Estructura de Datos y Algoritmos

Complejidad algorítmica

VIGILADA MINEDUCACIÓN

Facultad de
Ingeniería

Universidad de La Sabana




```
fun suma_arreglo (a: array [0:N) of int)  ret w: int

var i : int

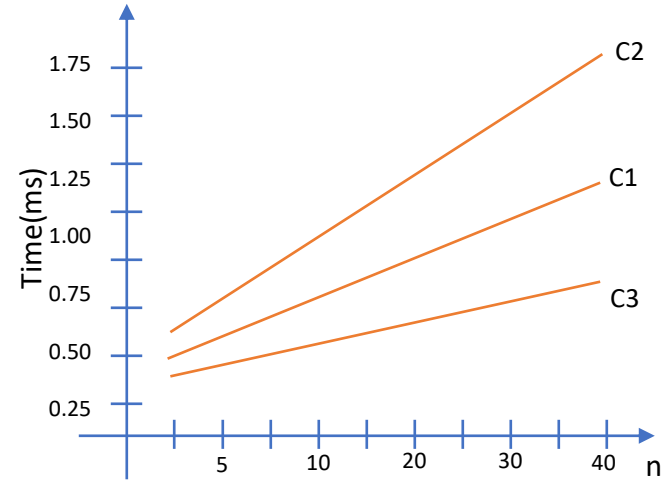
{Q: 0≤N}
w,i:=0,0

do i<N → w:=w+a[i];
        i:= i+1;
od

ret w
{R: w = (∑k|0≤k<N:a[k])}
```

¿Complejidad en tiempo?

¿Cuánto tiempo le toma a esta función correr?



¿Cómo crece el tiempo de ejecución de esta función?

Complejidad Tiempo: una manera de mostrar como el tiempo de ejecución crece en función de la entrada



Análisis de eficiencia algorítmica

- Necesitamos técnicas que nos permitan comparar la eficiencia de los algoritmos sin implementarlos. Independientes del lenguaje y de la máquina donde son ejecutados.
- Nuestras dos herramientas más importantes son:
  - El modelo de computación RAM
  - El análisis asintótico de la complejidad computacional.
- La evaluación del rendimiento algorítmico hace uso de la notación "Big Oh", que resulta esencial para comparar algoritmos y diseñar mejores.



Random Access Machine

- Computadora hipotética RAM: Random Access Machine.
- Bajo este modelo de computación, nos enfrentamos a una computadora donde:
  - Cada operación simple (+, *, -, =, if) toma exactamente un **paso** de tiempo.
  - Los bucles y las llamadas a subrutinas no son operaciones simples. Dependen del tamaño de los datos y del contenido de una subrutina. “Ordenar” no es una operación de un solo paso.
  - Cada acceso a la memoria toma exactamente un paso de tiempo. Además, tenemos tanta memoria como necesitamos.
- **Medimos el tiempo de ejecución de un algoritmo contando el número de pasos.**
- **Si asumimos que nuestra RAM ejecuta un número determinado de pasos por segundo, este recuento de operaciones se convierte naturalmente en el tiempo de ejecución real.**



Ejemplo

Sigamos la notación: Asignación (c_1),Comparaciones (c_2), +,*,-,/ (c_3)

```
fun suma_arreglo (a: array [0:N) of int)  ret w: int

var i : int

{Q: 0≤N}
w:=0
i:=0

do i<N →
l1  w:=w+a[i];
    i:= i+1;
od

ret w
{R: w = (∑k|0≤k<N:a[k])}
```

$2c_1$

$T(do) = n(c_2 + T(l_1))$

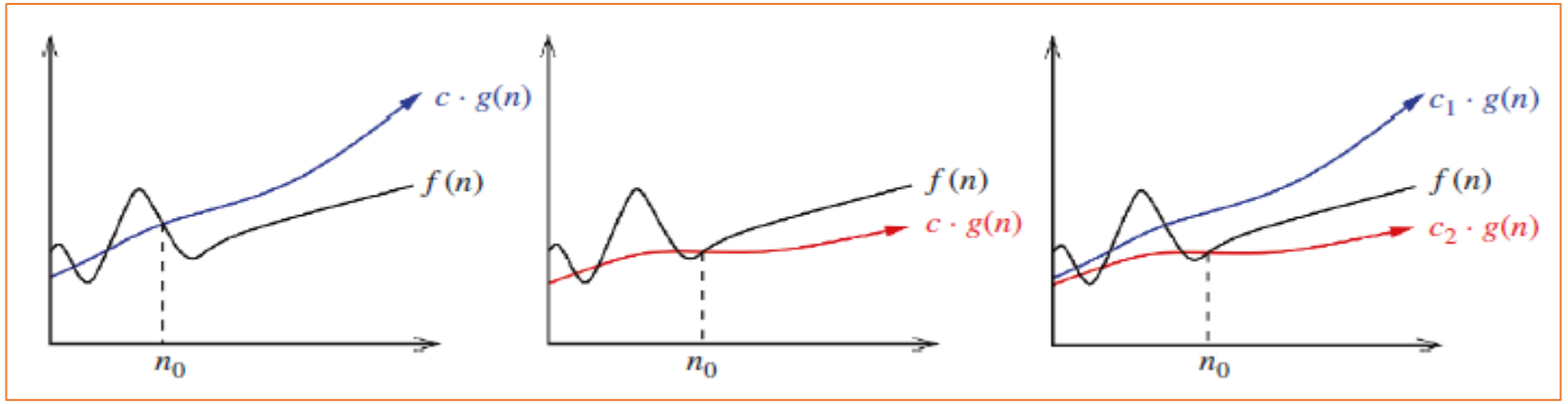
$T(l_1) = 2c_1 + 2c_3$

$$\boldsymbol{T(n) = n(c_2 + 2c_1 + 2c_3) + 2c_1}$$



Notación Big O

- $f(n) = O(g(n))$ significa que $c \times g(n)$ es un límite superior de $f(n)$.
  - Por tanto, existe una constante c tal que $f(n) \leq c \cdot g(n)$ para todo n suficientemente grande (es decir, para todo $n \geq n_0$, para alguna constante n_0).
- $f(n) = \Omega(g(n))$ significa que $c \times g(n)$ es un límite inferior de $f(n)$.
  - Por tanto, existe una constante c tal que $f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$.
- $f(n) = \Theta(g(n))$ significa que $c_1 \times g(n)$ es un límite superior de $f(n)$ y $c_2 \times g(n)$ es un límite inferior de $f(n)$ para todo $n \geq n_0$.
  - Por tanto, existe una constante c_1 y c_2 tal que $f(n) \leq c_1 \cdot g(n)$ y $f(n) \geq c_2 \cdot g(n)$ para todo $n \geq n_0$.

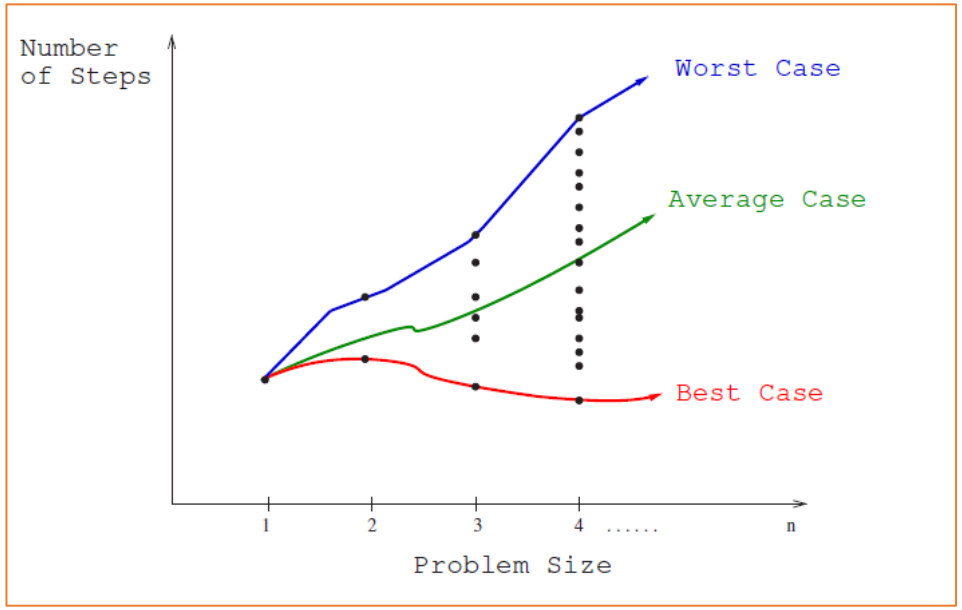


No nos preocupan los valores pequeños de n, cualquier cosa a la izquierda de n_0.



Complejidad del Peor, mejor y caso promedio

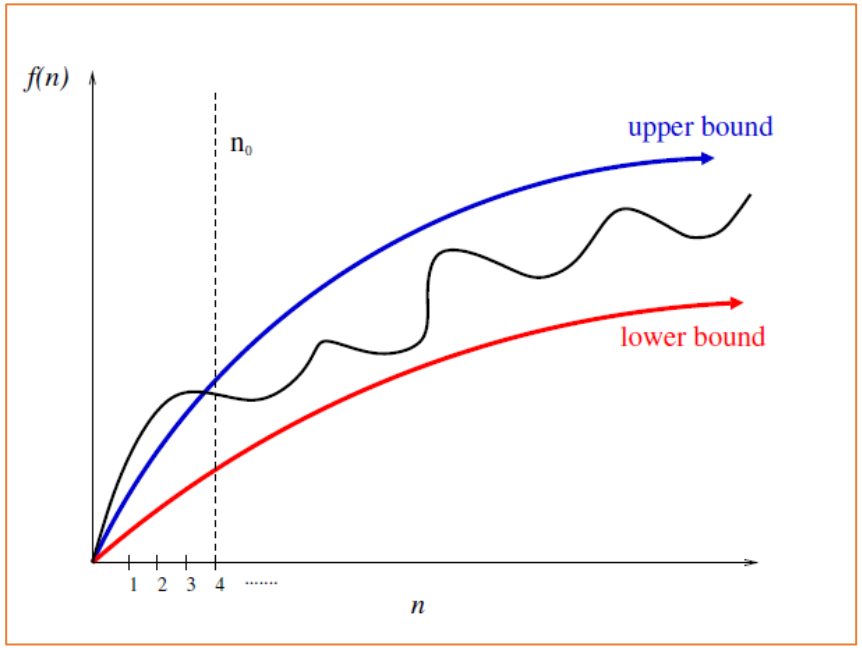
- La **complejidad del peor caso** de un algoritmo es la función definida por el número máximo de pasos dados en cualquier instancia de tamaño n.
- La complejidad del algoritmo en el **mejor de los casos** es la función definida por el número mínimo de pasos dados en cualquier instancia de tamaño n.
- La complejidad del **caso promedio** o el tiempo esperado del algoritmo, que es la función definida por el número promedio de pasos en todas las instancias de tamaño n.





Complejidad del Peor, mejor y caso promedio

- En términos generales, usaremos la complejidad del peor de los casos como nuestra medida preferida de la eficiencia del algoritmo.
- El análisis del peor de los casos es generalmente fácil de hacer y "por lo general" refleja el caso promedio.
- **¡Un análisis exacto es difícil!!.** El mejor, el peor y el promedio de los casos son difíciles de tratar porque los detalles precisos de la función suelen ser complicados.

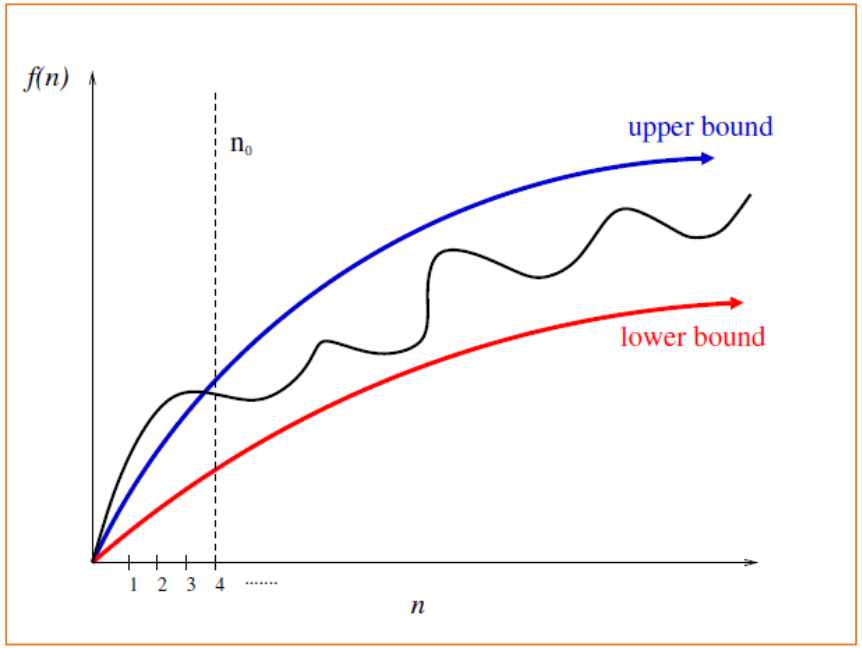


$$T(n) = 12754n^2 + 4353n + 834 \lg_2 n + 13546$$



Complejidad del Peor, mejor y caso promedio

- Es más fácil hablar de los límites superior e inferior de la función. La notación asintótica ($O; \Theta; \Omega$) es lo mejor que podemos hacer en la práctica con funciones de complejidad.



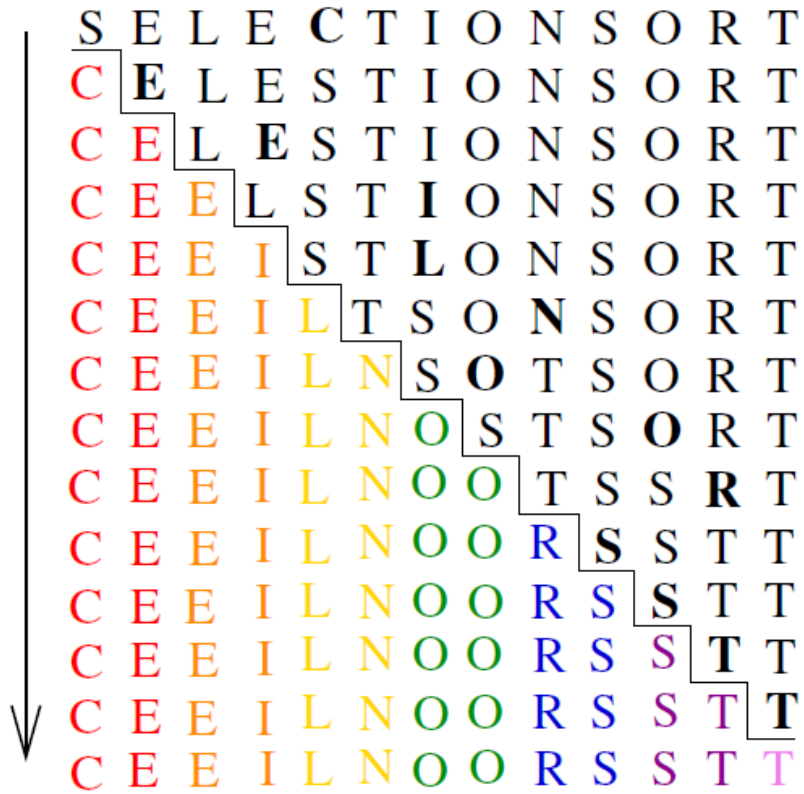


Actividad

```
void selection_sort(item_type s[], int n) {
    int i, j;    /* counters */
    int min;     /* index of minimum */

    for (i = 0; i < n; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (s[j] < s[min]) {
                min = j;
            }
        }
        swap(&s[i], &s[min]);
    }
}
```

Sigamos la notación: Asignación (c_1),Comparaciones (c_2), +,*,-,/ (c_3)





Selection Sort

Sigamos la notación: Asignación (c_1),Comparaciones (c_2), +,*,-,/ (c_3)

```
void selection_sort(item_type s[], int n) {
    int i, j;    /* counters */
    int min;     /* index of minimum */

    for (i = 0; i < n; i++) {
        min = i;
        for (j = i + 1; j < n; j++) {
            if (s[j] < s[min]) {
                min = j;
            }
        }
        swap(&s[i], &s[min]);
    }
}
```

l_1 l_2

$c_1 + n(c_2 + c_1 + c_3 + T(l_1))$

c_1

$c_1 + c_3 + \sum_{i=0}^{n-1} n - i - 1(c_2 + c_1 + c_3 + T(l_2))$

c_2

c_1

$T(swap) \cong c_{swap}$

$$T(n) = c_1 + n(c_2 + c_1 + c_3 + (2c_1 + c_3 + c_{swap} + \sum_{i=0}^{n-1} n - i - 1(2c_2 + 2c_1 + c_3)))$$

$$T(n) = k_1 + n(k_2 + \sum_{i=0}^{n-1} n - i - 1(k_3)))$$

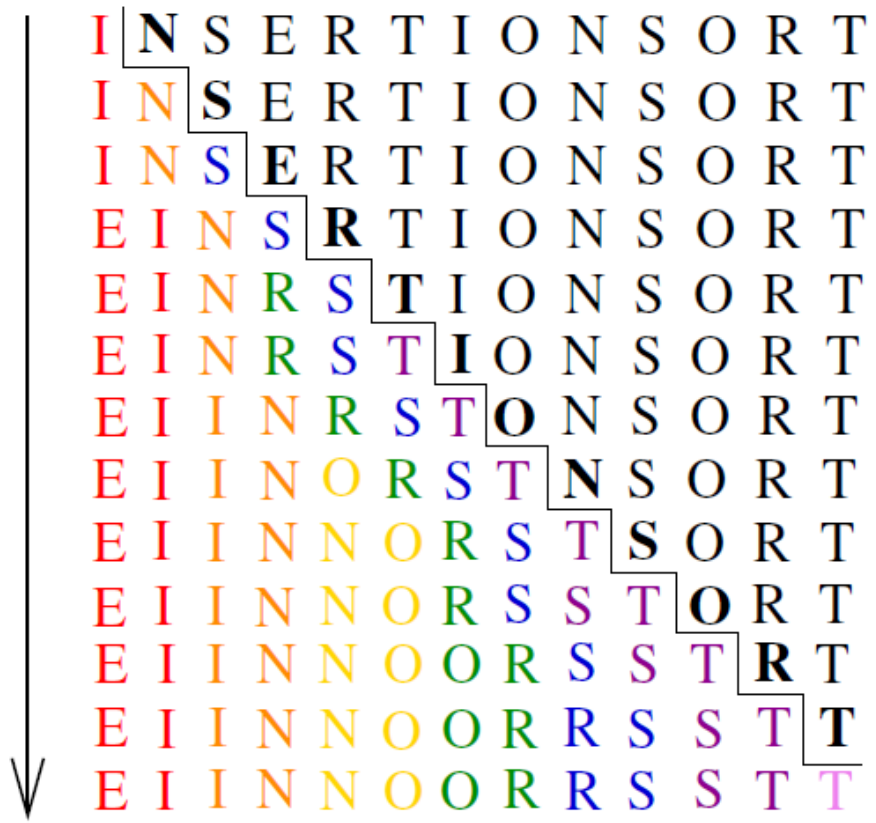


Actividad

```
void insertion_sort(item_type s[], int n) {
    int i, j;     /* counters */

    for (i = 1; i < n; i++) {
        j = i;
        while ((j > 0) && (s[j] < s[j - 1])) {
            swap(&s[j], &s[j - 1]);
            j = j-1;
        }
    }
}
```

Sigamos la notación: Asignación (c_1),Comparaciones (c_2), +,*,-,/ (c_3)





Análisis

- Mejor de los casos
- Peor de los casos

n

$\leq n-1$

```
void insertion_sort(item_type s[], int n) {
    int i, j;    /* counters */

    for (i = 1; i < n; i++) {
        j = i;
        while ((j > 0) && (s[j] < s[j - 1])) {
            swap(&s[j], &s[j - 1]);
            j = j-1;
        }
    }
}
```

Podemos asumir una cota de ejecución superior ajustada del while como n.

Por lo tanto, n llamados a un ciclo anidado que toma a lo mucho n pasos en cada llamada es $O(n^2)$.

Adicionalmente vimos que en el peor caso es como mínimo ($\approx \frac{n^2}{2}$) por lo tanto podemos afirmas que $\Omega(n^2)$.



Dominancia asintótica

n $f(n)$	$\lg n$	n	$n \lg n$	n^2	2^n	$n!$
10	0.003 μs	0.01 μs	0.033 μs	0.1 μs	1 μs	3.63 ms
20	0.004 μs	0.02 μs	0.086 μs	0.4 μs	1 ms	77.1 years
30	0.005 μs	0.03 μs	0.147 μs	0.9 μs	1 sec	8.4×10^{15} yrs
40	0.005 μs	0.04 μs	0.213 μs	1.6 μs	18.3 min	
50	0.006 μs	0.05 μs	0.282 μs	2.5 μs	13 days	
100	0.007 μs	0.1 μs	0.644 μs	10 μs	4×10^{13} yrs	
1,000	0.010 μs	1.00 μs	9.966 μs	1 ms		
10,000	0.013 μs	10 μs	130 μs	100 ms		
100,000	0.017 μs	0.10 ms	1.67 ms	10 sec		
1,000,000	0.020 μs	1 ms	19.93 ms	16.7 min		
10,000,000	0.023 μs	0.01 sec	0.23 sec	1.16 days		
100,000,000	0.027 μs	0.10 sec	2.66 sec	115.7 days		
1,000,000,000	0.030 μs	1 sec	29.90 sec	31.7 years		



Implicaciones de la dominancia

- Los algoritmos exponenciales crecen extremadamente rápido.
- Los algoritmos cuadráticos se vuelven inútiles en o antes de 1,000,000.
- O (n log n) es posible hasta aproximadamente mil millones.
- O (log n) es magia.

Facultad de
Ingeniería



Universidad de
La Sabana